UMHACKATHON 2026 · DOMAIN 1 · AI SYSTEMS & AGENTIC WORKFLOW AUTOMATION

Quality Assurance.

Risk anticipation, mitigation, and the test strategy that proves the GLM-critical path works end-to-end.

TEAM · OPUS MAGNUM

Oybek Odilov
Al-Bazeli Gameel Abduljalil
Lee Yih Shen
Samandar Erkinjonov
Abdugaffor Odilov

ONBOARD

POWERED BY Z.AI GLM · REGULATED ONBOARDING ENGINE

QATD · QUALITY ASSURANCE & TEST APRIL 2026 SUBMISSION RELEASE

1. QA Philosophy

Two non-negotiables shape every test we wrote:

1. **GLM is the system. Stub it and the system is meaningless.** Tests cannot prove the product works by mocking the model — they have to either run a real GLM call (gated by an API key) or assert structural invariants about the *audit trail* the model leaves behind.
2. **The audit trail is the test fixture.** `step_runs.decision_log` is persisted JSON. We diff its shape against snapshots so silent GLM drift surfaces as a test failure, not a demo embarrassment.

Everything else flows from those two rules.

2. Test Strategy

2.1 Three layers, one promise

LAYER	WHAT IT COVERS	LIVES IN
Unit	Pure functions (template compiler, RunState reducers, profile builder, OCR fallbacks, decision-log redaction)	`backend/tests/test_template_compile.py`, `test_processing.py`, `test_workflow_minimal.py`
Integration (HTTP)	Every router exercised against a real SQLite + a stubbed GLM client	`backend/tests/test_api_minimal.py`
Live GLM smoke	Capability matrix on the actual `ilmu-glm-5.1` model	`backend/scripts/glm_smoke.py`, `backend/tests/test_glm_smoke.py`

The promise: every PR that touches a primitive runs all three layers. CI gates on layer 1 + 2 always; layer 3 runs when `GLM_API_KEY` is present.

2.2 Why not mock GLM in integration tests

We made the explicit choice to inject a *deterministic transport stub* rather than a *model mock*: the stub returns canned `chat.completions` payloads keyed by tool name, so `call_tool("verify_against_criteria", ...)` returns a real-shape `tool_calls` envelope with one of three pre-canned verdicts. This means:

- The primitive's argument-parsing path is exercised (parsed JSON, tool_choice contract, retry-on-malformed branch).
- The `Decision` sink, the runner's snapshot, and the API surface are all hit by the same shape they'd see from a live call.

- A swap to a live model in CI is one env var, not a refactor.

A "model mock" — patching `GLMClient.call_tool` to return parsed Python — would skip every part of the system under test except the primitive's own happy path. We rejected it.

2.3 What we deliberately don't test

- **Network-level retry behaviour.** `tenacity` and the OpenAI SDK's own retry machinery are exercised by their maintainers; we don't simulate 429s in our suite.
- **LangGraph internals.** We test the compiler's output shape and the runner's checkpoint behaviour, not LangGraph's edge-routing.
- **3D scene rendering.** A visual smoke test on the published page is in the demo rehearsal checklist (§7), not in the automated suite.
- **Authentication.** Auth is mocked in MVP scope; testing a stub is theatre.

3. Test Coverage Matrix

TEST FILE	COVERS	GLM LIVE?	RUNS IN CI?
`test_glm_smoke.py`	The six capabilities in the TAD's §2.3 matrix (chat, JSON, tools, streaming, reasoning_effort, vision)	✓	When `GLM_API_KEY` set
`test_api_minimal.py`	Full HTTP loop: create city → create template → create building → start → submit form → upload doc → verify → publish. End-to-end.	✗ (stub)	✓
`test_workflow_minimal.py`	Primitive registry wiring; `RunState` reducers behave (no accidental erasure); `awaiting_user` pause/resume	✗	✓
`test_template_compile.py`	Template JSON → `StateGraph`: every primitive in the registry compiles; unknown primitives raise `ValueError`; the compiler is a pure function	✗	✓
`test_processing.py`	`pypdf` text-layer extraction, `pytesseract` OCR fallback when text is empty, image-extraction stub when the `[ocr]` extra is absent	✗	✓
`test_upload_compliance.py`	Compliance primitive in isolation: extraction + verification + decision-log shape; the re-upload loop on failure	✗ (stub)	✓

Acceptance bar for a primitive change

Before merging a change to any primitive in `backend/app/workflow/primitives/`:

1. Its dedicated test file passes (or a new one exists if the primitive is new).
2. `test_api_minimal.py` still runs the full template to `publish` without manual intervention.
3. `test_workflow_minimal.py` still validates the registry resolves the primitive.
4. `glm_smoke.py` runs cleanly against `ilmu-glm-5.1` for the capability the primitive depends on.

4. Risk Register & Mitigations

The risk register is structured as: *what can break*, *how it would manifest in the demo*, *how the system is hardened against it*, *how we'd catch it before judges do*.

4.1 Model & API risks

#	RISK	DEMO FAILURE MODE	MITIGATION IN CODE	VERIFICATION
R1	GLM rate-limited mid-demo (429)	Step stalls, owner sees a spinner	`GLMClient._chat` : 4× exponential backoff with jitter	`test_glm_smoke.py` confirms retry on simulated 429
R2	GLM 5xx	Same as R1	Same retry loop	Same
R3	GLM returns malformed tool arguments (`json.JSONDecodeError`)	Step crashes the run	`call_tool` catches, returns `{_error, _raw}` ; primitive marks `failed`	`test_upload_compliance.py` includes a malformed-args fixture
R4	GLM hallucinates `pass` without evidence	A bad doc gets approved	Verification primitive rejects empty-evidence verdicts and re-prompts once, then marks `failed`	Manual review of seeded fixtures + decision-log diff
R5	GLM behaviour drifts (prompt regression)	Verdicts silently change between runs	Decision-log snapshot tests — shape diff fails the build	`test_api_minimal.py` snapshot
R6	Vision unsupported on `ilmu-glm-5.1`	Image docs return empty	OCR fallback (`pdf2image` + `pytesseract`); `decision_log` marks `ocr_used=true`	`glm_smoke.py` flags it as `✗` ; `test_processing.py` covers the fallback
R7	ILMU outage	Total demo blocker	`AI_MODEL` switch to OpenAI exists for **dev fallback only**; not in submission envelope. Mitigated by rehearsing on a stable network and pre-running the demo	Rehearsal + smoke run

#	RISK	DEMO FAILURE MODE	MITIGATION IN CODE	VERIFICATION
			path immediately before the live demo	

4.2 Workflow / state risks

#	RISK	DEMO FAILURE MODE	MITIGATION IN CODE	VERIFICATION
R8	Template references unknown primitive	Run crashes on start	`compile_template` raises `ValueError("unknown primitive: …")` before graph executes	`test_template_compile.py`
R9	LangGraph checkpoint corruption	Run cannot resume	Reseed clears checkpoint rows in place (no file unlink); rewind allocates a fresh thread per `RunState.rev`	`test_workflow_minimal.py` exercises pause/resume
R10	Concurrent ticks on same thread	Race, lost state	One graph compile per HTTP tick; thread id `{run_id}:{rev}` ; rewind allocates a new thread	Manual review
R11	Owner submits scanned PDF with no text layer	Silent empty extraction → false-pass verification	`processing/pdf.py` falls back to `pdf2image + pytesseract` ; empty extracts surface as "document unreadable — please re-upload"	`test_processing.py` covers both branches
R12	Cross-check surfaces trivial diffs (date format)	Owner sees noisy false-positive contradictions	Cross-check prompt has explicit exclusion rules; `severity ≤ "minor"` hidden from owner UI	Decision-log review on seeded run
R13	Autofill proposes a wrong value	Owner accepts a bad fix	UI requires explicit confirmation before applying; the original value is preserved in `decision_log`	Manual review on seeded fixtures
R14	StepForm's schema is missing a field name	Form fails to render	Renderer falls back to field index when `name` is missing	Integration test exercises a field-name-less template

4.3 Data & deployment risks

#	RISK	DEMO FAILURE MODE	MITIGATION IN CODE	VERIFICATION
R15	OCR binary missing on the host	Image extraction silently empty	`processing/pdf.py` and `image.py` log `ocr_used=false`; `[ocr]` extra is documented in install	Install-time check in CI
R16	Seed data missing on a fresh DB	Demo can't start	`just seed` creates admin, owner, Shah Alam city, template, Menara Demo building; `reseed_demo` re-runs without restarting backend	Manual on every fresh container
R17	Schema drift between SQLite and Postgres	Works in dev, fails in prod	SQLAlchemy 2.0 maps abstract types; JSON columns are JSONB on Postgres / TEXT on SQLite — same code	Render staging deploy before judging
R18	CORS misconfigured between Vercel + Render	Frontend can't reach backend	`render.yaml` wires the frontend origin into `ALLOWED_ORIGINS`	Pre-demo curl
R19	`.env` leaks `GLM_API_KEY` into a commit	Key revoked, demo dead	`.env*` is gitignored; `.env.example` ships variable names only	Pre-commit grep + manual review
R20	Render free-tier cold start	First demo step takes 30s	Pre-warm with a `/health` poll 60 s before the demo	Rehearsal

5. GLM-Specific Verification

The capability matrix from the TAD §2.3 is the GLM acceptance bar. We treat it as a **pre-flight check**, not a one-time discovery:

```
just smoke-glm
```

...runs `backend/scripts/glm_smoke.py` and writes `backend/scripts/glm_smoke_results.json`. Each capability is asserted independently. A single failure on the smoke run gates a deploy.

The eight product tools (`draft_template` , `explain_field` , `extract_document` , `verify_against_criteria` , `cross_check_documents` , `propose_auto_fix` , `summarize_for_review` , `answer_visitor_question`) are covered by integration fixtures using the deterministic stub (§2.2). Adding a ninth tool requires:

1. A schema in `backend/app/glm/tools.py`.
2. A system prompt in `backend/app/glm/prompts.py`.
3. A primitive (or extension) that calls it.
4. A new entry in the integration suite's stub map.
5. A smoke probe demonstrating the tool returns a parseable structured response from `ilmu-glm-5.1`.

6. Test Execution

6.1 Local

```
just test        # backend/tests, ~5 s without live GLM
just smoke-glm   # ~30 s against the real model
```

`pytest` is configured under `backend/pyproject.toml` (`asyncio_mode = "auto"`, `testpaths = ["tests"]`). Live-GLM tests are decorated with `@pytest.mark.skipif(not os.getenv("GLM_API_KEY"))` so the suite stays green on CI without a key.

6.2 Manual demo rehearsal

The demo path is itself a regression test. Before each rehearsal:

1. `just seed` to ensure a clean database.
2. Walk the three-minute script: admin authoring, owner upload + autofill, contradiction surfacing + accept fix, publish, visitor chat with camera nav.
3. Open the decision-log drawer at every step; confirm each `step_runs.decision_log` entry is present and well-formed.
4. Verify the published building's 3D scene renders and the visitor chat refuses an out-of-scope question.

A rehearsal failure blocks the next deploy.

7. Demo-Readiness Checklist

CHECK	WHERE
`GLM_API_KEY` is set and validates against `https://api.ilmu.ai/v1`	Backend env / `/health` returns model name
Database schema is fresh and seeded	`just seed` exits clean; admin and owner exist
Seeded Shah Alam template has the six primitives in order	UI: admin → cities → Shah Alam
Demo documents (Bomba cert, MBSA zoning, deed, listing pack) are present	`backend/app/seed/demo_docs/`
`/health` returns `{"ok": true, "model": "ilmu-glm-5.1"}`	curl
Pre-warm tick: `/buildings` lists the published demo building	Visitor surface
Frontend is reachable and CORS-clean from the Render origin	Browser network tab
The smoke run is green within the last hour	`backend/scripts/glm_smoke_results.json` mtime

8. Known Open Issues

- **Render free-tier cold start.** First request after idle can take 20–30 s. We pre-warm before the demo; long-term mitigation is the Standard tier.
- **No load testing.** Concurrent owner runs are out of MVP scope; the SSE endpoint's 500 ms poll would not survive a surge.
- **No frontend test suite.** UI behaviour is exercised only by manual rehearsal and the integration tests' API contract. A Playwright smoke layer is the natural follow-up.
- **OCR throughput on long scanned PDFs.** `pdf2image` per page is sequential; multi-page scans can take 5–15 s. Owners see live status so the latency is never silent, but throughput is bounded.

See the SAD for runtime data flows and the TAD for the technology choices these tests are validating against.